

Lab 11

DSA

April 17, 2026

Introduction: Heap Sort

Given an array $A[1 : n]$, Heap Sort works as follows:

1. Call BUILD-MAX-HEAP to build a max-heap.
2. Move the maximum element to the end by exchanging $A[1]$ with $A[n]$.
3. Reduce the heap size by 1.
4. Call MAX-HEAPIFY at the root to restore the heap property.

The heapsort algorithm then repeats this process (Steps 2 to 4) for the max-heap of size $n-1$ down to a heap of size 2. The same can be seen from the pseudocode below:

HEAP-SORT(A)

```
1  BUILD-MAX-HEAP( $A$ )
2  for  $i = A.length$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

Coding Demo

In this lab, the teaching assistants will demonstrate how to implement the core heap operations required for Heap Sort.

1. Max-Heap Representation

A heap can be efficiently represented using an array. If a node is stored at index i :

- Left child is at index $2i + 1$
- Right child is at index $2i + 2$
- Parent is at index $\lfloor (i - 1)/2 \rfloor$

2. Max-Heapify Function

The **Max-Heapify** function assumes subtrees $\text{left}(i)$ and $\text{right}(i)$ are max-heaps, but $A[i]$ might be smaller than its children, thus violating the max-heap property. Max -Heapify lets the value $A[i]$ "flow down" in the max-heap so that the subtree rooted at i obeys the max-heap property.

```

1 void maxHeapify(int A[], int n, int i) {
2     int largest = i;
3     int left = 2*i + 1;
4     int right = 2*i + 2;
5
6     if (left < n && A[left] > A[largest])
7         largest = left;
8
9     if (right < n && A[right] > A[largest])
10        largest = right;
11
12    if (largest != i) {
13        int temp = A[i];
14        A[i] = A[largest];
15        A[largest] = temp;
16
17        maxHeapify(A, n, largest);
18    }
19 }

```

The Max-Heapify function compares the element at index i with its left and right children and identifies the largest among them. If one of the children is larger than the current node, a swap is performed to move the largest element to the root of the subtree. The function is then recursively applied to the affected subtree to ensure that the max-heap property is maintained throughout.

3. Build-Max-Heap Function

The **Build-Max-Heap** function converts an unsorted array into a max heap by calling Max-Heapify in a bottom-up manner.

```

1 void buildMaxHeap(int A[], int n) {
2     for (int i = n/2 - 1; i >= 0; i--) {
3         maxHeapify(A, n, i);
4     }
5 }

```

The Build-Max-Heap function starts from the last non-leaf node (at index $n/2-1$) and moves upward to the root, calling Max-Heapify at each step. Since leaf nodes already satisfy the heap property, this bottom-up approach ensures that all subtrees are converted into valid max-heaps, resulting in the entire array representing a max heap.

Exercise

Using the above two functions, implement the heapsort algorithm in C or C++.

Your program must first build a max heap from the input array, to perform sorting using Heap Sort, and then print the sorted array. [CO1, CO2, CO3, CO4]

Sample Input

```

Enter number of elements: 5
Enter elements: 4 10 3 5 1

```

Sample Output

Sorted array: 1 3 4 5 10

Rubric (10 Marks)

- Correct implementation of Max-Heapify (3 marks)
- Correct Build-Max-Heap logic (3 marks)
- Correct Heap Sort implementation (4 marks)